

Procedimento para resolver problemas no beecrowd Online Judge

Sites do tipo Online Judge são uma importante ferramenta para alunos de disciplinas de programação. Este material indica um procedimento padrão para o estudante que deseja se aprimorar em algoritmos e técnicas de programação por meio do *beecrowd Online Judge* (<https://judge.beecrowd.com/pt>). Com algumas adaptações menores, serve também para o *Codeforces* (<https://codeforces.com/>), *DM::OJ* (<https://dmoj.ca/>), *Neps Academy* (<http://neps.academy/>), *LeetCode* (<https://leetcode.com/>), *CodeChef* (<https://www.codechef.com/>), *Spoj* (<https://www.spoj.com/>), *HackerRank* (<https://www.hackerrank.com/contests/practiceieeepl/challenges/the-beginning>), *CodeWars* (<https://www.codewars.com/>) e outros sites de mesmo tipo.

A organização do trabalho é um aspecto essencial, e independe da plataforma utilizada. Ter critérios claros de organização e padrões próprios facilitam enormemente o trabalho de resolver problemas e tornam o estudante mais produtivo, tanto ao fazer listas de exercício como ao participar de competições. Algumas recomendações a seguir têm a ver com esse aspecto organizacional, outras com a natureza do trabalho do programador, outras com particularidades do trabalho nas disciplinas de programação da Fatec-SO.

1. Defina um padrão geral de nomenclatura: você precisará criar pastas, arquivos fonte, arquivos de entradas de dados, arquivos de saída de dados, etc. Defina de antemão um padrão e seja consistente. Em geral os problemas são mais facilmente referenciados pelo seu número no site do beecrowd, então utilizar esse número na composição dos nomes de pastas e arquivos relacionados é uma boa ideia.
2. Crie uma pasta específica para cada problema que for resolver. Nessa pasta você deve colocar o arquivo fonte, os arquivos de entradas e de saídas e outros arquivos que criar ou utilizar para resolver o problema.
3. Com base no enunciado do problema, estude os casos de teste originais e crie outros casos. Coloque esses casos de teste em arquivo em formato ASCII, para poder utiliza-los ao testar o programa futuramente. Calcule as saídas esperadas para cada caso de teste. Depois, utilize a opção uDebug do beecrowd para determinar a resposta correta esperada pelo site. Compare as suas respostas geradas manualmente com aquelas fornecidas pelo site e verifique eventuais divergências. Perca o tempo que for necessário nesta etapa, discuta com colegas, monitores e professor até conseguir entender a saída exata para cada caso de teste. É nesta etapa que você está efetivamente resolvendo o problema (programando), pois precisa entender as entradas e saídas esperadas e o trabalho requerido para produzir os resultados. Depois que esta etapa tiver sido concluída, bastará a você codificar a solução em C, pois o problema já estará resolvido em sua cabeça. Programar é pensar em uma solução, codificar é apenas traduzir essa solução para uma linguagem específica. Programe primeiro, codifique depois.
4. Codifique o seu programa, de acordo com o raciocínio desenvolvido ao resolver os casos de teste. É interessante ter um esqueleto básico de programa para ser usado como ponto de partida. Nesse esqueleto deixe todos os `#include` que costumamos utilizar (`stdio.h`, `stdlib.h`, `string.h`, `math.h` e `ctype.h`, pelo menos), um cabeçalho para uma descrição geral do programa e de detalhes relevantes sobre a abordagem utilizada para resolver o problema, a data, etc.

Procedimento para resolver problemas no beecrowd Online Judge

5. Compile o programa e teste com os casos de teste do enunciado e também com os casos de teste que você desenvolveu. Procure obter uma compilação limpa, sem mensagens de erro ou de advertência. SEMPRE execute o programa em linha de comando (no *prompt* do *cygwin*, do Linux ou do DOS) fazendo o redirecionamento da entrada, para ser mais produtivo. Quando os resultados produzidos pelo programa estiverem corretos, redirecione também a saída, para verificar eventuais problemas de formatação de saída (quebras de linha sobrando ou faltando, espaços em branco iniciais ou finais nas linhas, etc).
6. Se o programa atende ao enunciado e apresenta resultados corretos tanto para os casos de teste do enunciado como para os casos de teste que você mesmo gerou, submeta seu programa no site do beecrowd e aguarde o resultado.
7. Caso a submissão não tenha sido aceita, analise o erro e faça as modificações necessárias. Se a resposta for *Wrong Answer*, tente criar novos casos de teste para identificar o que pode estar errado em seu código. Se você ainda não entende direito o significado das mensagens com o resultado das submissões, entre na opção Submissões do menu principal e escolha o ícone “Respostas: O que é isto significa?” no lado esquerdo da tela para uma rápida explicação sobre o tema.
8. Resista à tentação de ficar olhando para o código de outras pessoas enquanto você ainda não conseguiu resolver o problema, pois isso só vai atrapalhá-lo. Na seção Forum do beecrowd você encontra discussões sobre cada problema, exemplos de casos de teste que você pode utilizar para testar seu programa e também, infelizmente, códigos fonte de outras pessoas. Não fique olhando esses códigos enquanto não conseguir um Accepted para a sua solução. DEPOIS que conseguir resolver, então poderá ser interessante observar a solução de outras pessoas, pois daí terá um termo de comparação com o que você fez.